

CM3 - Data Orchestration

Apache Airflow & Production Pipeline Management

Théo Burdinat

theo.burdinat1@epfedu.fr

May 20, 2026

CM2 Recap: Beyond Writing Code

CM2 Recap: Important Concepts to Remember

What CM2 was really about

- Not just writing optimized Spark code.
- Understanding **physical execution plans** and **runtime behavior**.
- Mastering **join strategies**, **skew mitigation**, and **AQE**.

Key skills to remember

- Reading **execution plans**
- **Broadcast vs Sort-Merge joins**
- **Partition** strategy
- **Skew** detection and mitigation
- **Memory** management

An important reality

Writing an optimized Spark job is an essential foundation, representing about **50% of the work**.

The remaining **50%** focuses on ensuring reliable execution in production environments.

A realistic production scenario

When things don't go as planned

Consider a perfectly optimized Spark job from CM2 that processes daily financial revenue data.

The scenario:

- The job is scheduled to run at 3 AM every day.
- Today, the source CSV file arrives 2 hours late (vendor delay).
- Your job crashes with `FileNotFoundException`.
- At 8 AM, the CEO opens the revenue dashboard: **completely empty**.
- Your phone rings.

The question

Your Spark code was perfect. So why did the system fail?

From Cron to Modern Orchestrators

Why cron has limitations for modern Data Engineering

What cron does

- Triggers a command at a specific time
- Simple, ubiquitous on Unix systems
- Works perfectly for simple tasks

What cron cannot do

- **Dependencies:** Job B must wait for Job A to succeed, not just "run after Job A's scheduled time"
- **State:** No centralized history; if the server reboots, execution state is lost
- **Retry logic:** No automatic retry on failure
- **Backfill:** Rerunning a month of history is a nightmare

The root problem

Cron is **time**-based, not **dependency**-based. Modern data pipelines need the latter.

What is an orchestrator?

The conductor analogy

An orchestrator is like a conductor in a symphony.

- The conductor **does not play any instrument** (does not process data).
- The conductor ensures **everyone plays in rhythm, in the correct order**.
- If a musician is absent, the conductor **alerts the audience**.

Orchestrator = Traffic Controller

Airflow coordinates jobs. It does not replace Spark, Snowflake, or your SQL database.

Key insight

Airflow is **infrastructure**, not **computation**.

Heavy data processing (Spark, BigQuery, Snowflake) happens **outside** Airflow.

Airflow just **submits jobs** and **monitors their state**.

Orchestrators in the ecosystem

Tool	Strength	Typical use case
Apache Airflow	Python-native, flexible, large community	General-purpose data pipelines
Prefect	Modern API, dynamic DAGs, cloud-native	Event-driven and parameterized workflows
Dagster	Asset-centric, strong typing, testing-first	Data platform teams with heavy governance
Kubernetes CronJobs	Simple, container-native	Basic scheduled tasks in K8s environments

CM3 focus: Apache Airflow

Airflow is the de facto standard in 2026. Learning Airflow teaches you orchestration principles that transfer to any tool.

Airflow Anatomy: Concepts & Architecture

The DAG: Directed Acyclic Graph

What is a DAG?

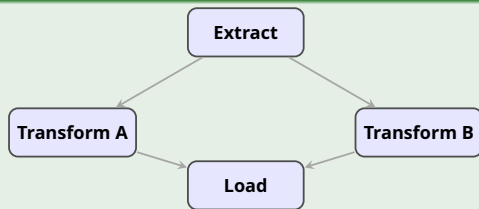
- **Directed:** Tasks flow in one direction (A → B → C)
- **Acyclic:** No loops. A pipeline has a clear start and end.
- **Graph:** Tasks are nodes; dependencies are edges.

Critical constraint

No cycles allowed. If Task A depends on Task B, and Task B depends on Task A, your pipeline is invalid.

This prevents infinite loops in production systems.

Simple DAG example



Parallel transforms, then final load.

DAGs are defined as Python code

Code as configuration

In Airflow, DAGs are **not** YAML or JSON config files.

DAGs are **Python scripts** that define the workflow structure.

Advantages

- Version control (Git)
- Programmatic logic (loops, conditionals)
- Easy integration with Python libraries
- Testable with standard Python tools

The catch: DAG files are parsed constantly

The Airflow Scheduler **parses every DAG file every 30 seconds** (default).

This means:

- Do NOT place heavy computation at the top level of your DAG file
- Do NOT make API calls or database queries outside of tasks
- Keep DAG file logic lightweight

We will cover this anti-pattern in detail later.

Operators vs Sensors: The two task types

Operators: "Do something"

Operators **execute an action**.

Examples:

- BashOperator: Run shell command
- PythonOperator: Run Python code
- SparkSubmitOperator: Submit Spark job
- S3ToRedshiftOperator: Load data from S3 to Redshift

Sensors: "Wait for something"

Sensors **wait for a condition**.

Examples:

- FileSensor: Wait for file on S3
- HttpSensor: Wait for API to return 200
- ExternalTaskSensor: Wait for DAG to complete

Sensors replace `sleep()` loops.

Two modes: `poke` or `reschedule`.

Real-world use case

Use a `FileSensor` to confirm input CSV has arrived on S3 before running Spark.

Airflow architecture: The four components

1. Scheduler

The brain of Airflow.

- Parses DAG files every 30 seconds
- Determines which tasks are ready to run
- Submits tasks to the Executor

2. Webserver

The UI for humans.

- Dashboard for monitoring DAG runs
- Trigger manual runs
- View logs and task history

3. Metadata Database

The source of truth.

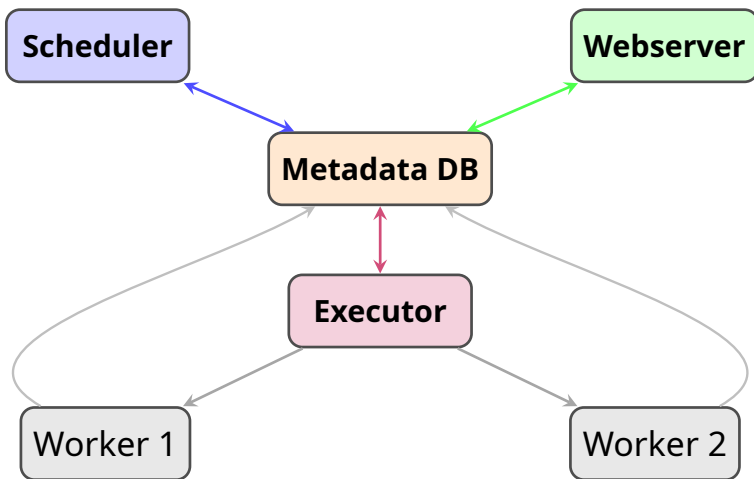
- Stores DAG definitions, task states, execution history
- Typically PostgreSQL or MySQL
- All Airflow components read/write from this DB

4. Executor

The worker coordinator.

- Runs tasks on workers (local, Celery, K8s)
- Handles task parallelism

Airflow architecture diagram



Architecture: How components interact (1/2)

The central role of the Metadata Database

The **Metadata Database** is the single source of truth for all Airflow state.

If corrupted or unavailable, Airflow cannot function.

All four components continuously read/write from this database.

Architecture: How components interact (2/2)

Typical workflow

- **Scheduler** parses DAG files every 30 seconds and writes task instances to the Metadata DB
- **Executor** reads ready tasks from the DB and dispatches them to available Workers
- **Workers** execute tasks and write results (success/failure, logs) back to the DB
- **Webserver** reads from the DB to display the UI and task status in real-time

Key insight

This architecture ensures that even if a component crashes, the state is preserved in the database and can be recovered.

Understanding Airflow Scheduling

Scheduling in production: The real challenge

A fundamental question

When you configure a daily pipeline to process "yesterday's data," a critical question arises:

When exactly should the pipeline run?

Naive approach

- Run at midnight (00:00)
- Process "today's" data
- Simple, intuitive

The problem

- Data arrives throughout the day
- Last records arrive at 23:59
- Running at 00:00 misses data!

Airflow's solution

Wait for the **complete data interval** before processing. This is the core of Airflow's scheduling model.

execution_date: A different mental model

Key concept

execution_date (or logical_date) represents **the start of the data interval** being processed, not when the DAG runs.

Concrete example

Daily DAG to process data for **January 1st, 2026**.

When does it run?

On **January 2nd at 00:00**, after January 1st is complete.

```
execution_date = 2026-01-01 00:00  
actual_run_time = 2026-01-02 00:00
```

Real-world scenario

Your pipeline aggregates daily sales from a transactional database.

If you run at 00:00 on January 1st, you miss the last transactions (23:59).

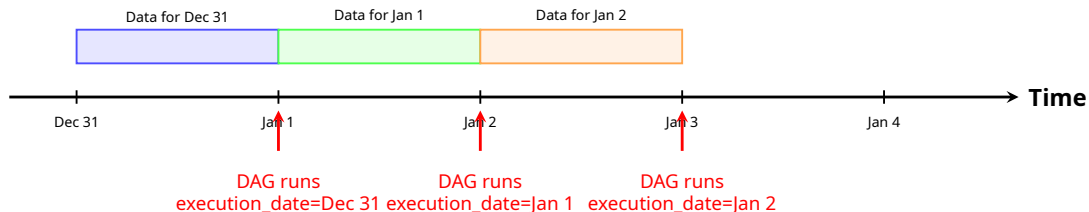
Airflow waits for the full day.

Common mistake

```
start_date = datetime(2026,  
1, 1) runs on Jan 2nd.
```

```
For Jan 1st: start_date =  
datetime(2025, 12, 31).
```

Scheduling timeline visualization



Key insight

The DAG for `execution_date = Jan 1` runs **at the end of Jan 1**, which is the **start of Jan 2**.

Catchup and Backfill: Time travel for data

What is catchup?

If your DAG's `start_date` is in the past and `catchup=True`, Airflow will automatically run **all missed intervals**.

Example:

- `start_date = 2025-01-01`
- Today is `2026-05-10`
- Airflow will run **495 daily DAG instances** to catch up

Danger zone

If `catchup=True` by accident on a year-old `start_date`, you might trigger thousands of runs.

When catchup is useful

Deploy DAG to backfill 6 months.

Set `start_date` 6 months ago, `catchup=True`.

Manual backfill

```
airflow dags backfill  
-s 2025-01-01 -e  
2025-06-01 my_dag
```

Idempotence: The golden rule of backfills (1/2)

Critical requirement

A DAG run for a given `execution_date` must produce **exactly the same result** whether run once or ten times.

Idempotent example (correct)

- Read data partitioned by date
- Write with mode ("overwrite")
- Use `execution_date` to filter

Running 10 times produces identical output.

Non-idempotent anti-pattern

- Write with mode ("append")
- Read "latest data" without filtering
- Use `datetime.now()` in logic

Running 10 times produces **duplicated data**.

Idempotence: The golden rule of backfills (2/2)

Connection to CM2

Partition strategy from CM2: Partitioning by date makes idempotent writes trivial.

Example: `s3://data/year=2026/month=01/day=01/`

Overwriting a single partition ensures idempotence.

Why idempotence matters

Without idempotence:

- Backfills duplicate data
- Retries corrupt datasets
- Manual reruns are dangerous

Golden rule: Design every pipeline to be safely rerunnable.

Common Pitfalls and Best Practices in Production

Common pitfall #1: Airflow is not a compute engine

A critical distinction

Airflow is an **orchestrator**, not a **data processor**.

Running heavy computation inside Airflow tasks can overload your Scheduler and impact your entire orchestration system.

Anti-pattern example

```
def process_data():  
    df = pd.read_csv("s3://data/5GB.csv")  
    result = df.groupby("user").sum()  
    result.to_parquet("s3://out/")
```

```
PythonOperator(  
    task_id="process",  
    python_callable=process_data  
)
```

This loads 5 GB into the Airflow worker's RAM.

Correct pattern

```
SparkSubmitOperator(  
    task_id="submit_spark_job",  
    application="/path/to/spark_job.py",  
    conn_id="spark_cluster"  
)
```

Airflow submits to Spark. Spark processes data.

Common pitfall #2: Top-level code in DAG files

A subtle performance issue

The Airflow Scheduler parses every DAG file **every 30 seconds**.

Any code at the **top level** runs every 30 seconds.

Anti-pattern example

```
import requests

# BAD: This runs every 30 seconds
api_data = requests.get("https://api.example.com/config")
.json()

with DAG("my_dag", ...):
    task = PythonOperator(...)
```

This creates 120 API calls per hour per DAG file. You DDoS your own API.

Correct pattern

```
import requests

def fetch_config():
    # GOOD: Only runs during task execution
    api_data = requests.get("https://api.example.com/
                             config").json()
    return api_data

with DAG("my_dag", ...):
    task = PythonOperator(
        python_callable=fetch_config
    )
```

Move expensive logic inside task callables.

Common pitfall #3: XComs for large data transfer (1/2)

What are XComs?

XComs (Cross-Communications) allow tasks to pass **small metadata** between each other.

Examples:

- Pass a file path from Task A to Task B
- Pass a processed date string
- Pass a record count (for validation)

XComs are stored in the **Metadata Database**.

What XComs are NOT for

Do NOT pass large data through XComs:

- CSV file contents
- JSON payloads > 1 MB
- Pandas DataFrames

Common pitfall #3: XComs for large data transfer (2/2)

Consequences of misusing XComs

XComs are limited to a few kilobytes. Exceeding this will:

- Slow down the Metadata Database
- Cause serialization failures
- Make the UI unresponsive

Correct pattern

Task A writes large data to S3, returns the S3 path via XCom. Task B reads from that path.

Key principle: Pass *references* to data, not the data itself.

Common pitfall #4: Ignoring retries and SLAs (1/2)

Transient failures are normal

In distributed systems, tasks fail temporarily:

- Network timeout
- API unavailability
- Worker reboot

Solution: Configure automatic retries.

Retry configuration

```
task = SparkSubmitOperator(  
    task_id="spark_job",  
    retries=3,  
    retry_delay=timedelta(minutes=5)  
)
```

Airflow retries 3 times with 5-min delay.

Common pitfall #4: Ignoring retries and SLAs (2/2)

SLAs: Business guarantees

An SLA (Service Level Agreement) defines **how long a task can run**.
If exceeded, Airflow alerts the team.

SLA configuration

```
task = SparkSubmitOperator(  
    task_id="critical_job",  
    sla=timedelta(hours=2)  
)
```

Team notified if task exceeds 2 hours.

Common pitfall #5: Insufficient observability (1/2)

The importance of monitoring

Without proper monitoring, failures can go unnoticed until they impact business operations.

Observability checklist

- **Email alerts** on task failure
- **Slack/PagerDuty integration**
- **Task logs** to centralized storage
- **Metrics dashboards** (Grafana, Datadog)

Alert configuration

```
default_args = {  
    "email": ["data-team@company.com"],  
    "email_on_failure": True,  
    "email_on_retry": False,  
}  
  
with DAG("critical_pipeline",  
        default_args=default_args,  
        ...):  
    ...
```

Failures trigger immediate email alerts.

Common pitfall #5: Insufficient observability (2/2)

Production reality

Teams integrate Airflow with incident platforms (PagerDuty, Opsgenie) to page on-call engineers.

Key observability metrics

- Task success/failure rate
- Task duration trends
- DAG execution delays
- Resource utilization (CPU, memory)

Proactive monitoring prevents production incidents.

Production Patterns and Best Practices

Best practice #1: Task design principles

Atomic tasks

Each task should do **one thing well**.

Bad: One task that extracts, transforms, and loads data.

Good: Three separate tasks (Extract, Transform, Load) with clear dependencies.

If Transform fails, you can retry it without re-extracting the data.

Idempotent tasks

Each task must be **safe to retry**.

- Use `overwrite` mode for writes
- Use `execution_date` for partitioning
- Avoid `INSERT` without `WHERE`

If a task is retried 5 times, the final result should be identical to running it once.

Early validation

Validate inputs early. If the source file is missing, fail immediately with a clear error message.

Best practice #2: Dynamic DAG generation (1/2)

The problem

You have 50 identical pipelines, one per country.

Bad: Copy-paste 50 DAG files.

Good: Generate DAGs programmatically.

Dynamic DAG example

```
COUNTRIES = ["FR", "DE", "US", "JP"]

for country in COUNTRIES:
    dag_id = f"pipeline_{country}"

    with DAG(dag_id, ...):
        task = SparkSubmitOperator(
            task_id=f"process_{country}",
            application=f"/jobs/process.py",
            application_args=[country]
        )
```

Best practice #2: Dynamic DAG generation (2/2)

Key insight

Since DAGs are Python code, you can use loops, functions, and external configuration files.

This level of flexibility is harder to achieve with purely declarative orchestrators.

Real-world pattern

Store the list of countries in a YAML config file:

`countries.yaml`

```
countries:  
- FR  
- DE  
- US
```

Read it in the DAG file and generate tasks dynamically.

Best practice #3: Separation of concerns (1/2)

DAG file = workflow logic only

The DAG file should define:

- Task dependencies ($A \rightarrow B \rightarrow C$)
- Scheduling parameters
- Retry/timeout configurations

The DAG file should **not** contain:

- Business logic
- Data transformations
- Complex SQL queries

Correct architecture

- **DAG file** (pipeline.py): workflow
- **Spark job** (transform.py): data logic
- **SQL scripts** (queries/): SQL
- **Config** (config.yaml): parameters

Best practice #3: Separation of concerns (2/2)

Why this matters

Changes to Spark jobs don't require DAG redeployment. This decouples orchestration from computation.

Benefits of separation

- DAGs remain stable, even when business logic changes
- Spark jobs can be tested independently
- Code is more maintainable and easier to debug
- Team can version and deploy jobs separately

Call pattern

```
SparkSubmitOperator(application="/path/to/transform.py", ...)
```

Best practice #4: Testing DAGs (1/2)

Unit testing DAG structure

You can test that your DAG is valid **without running it**.

```
import pytest
from airflow.models import DagBag

def test_dag_loads():
    dagbag = DagBag()
    assert len(dagbag.import_errors) == 0

def test_task_count():
    dag = dagbag.get_dag("my_pipeline")
    assert len(dag.tasks) == 5
```

Integration testing

Test the full DAG on a small dataset:

- Use Airflow's test mode:
airflow tasks test
- Run the DAG on a staging environment
- Validate output against expected results

Test thoroughly before deploying to production.

Best practice #4: Testing DAGs (2/2)

CI/CD for DAGs

Mature data teams treat DAGs like application code:

- DAGs are versioned in Git
- Pull requests require DAG validation tests to pass
- Automated deployment to staging, then production

Typical CI/CD pipeline

- Developer pushes DAG changes to a feature branch
- CI runs: DAG syntax validation, unit tests, linting
- If tests pass, DAG is deployed to staging
- Manual approval triggers production deployment
- Automated rollback if production tests fail

Real-World Use Case: Building a Production Pipeline

Scenario: Daily revenue reporting pipeline

Business requirement

Every day, aggregate revenue from the previous day and load it into a dashboard.

Data sources:

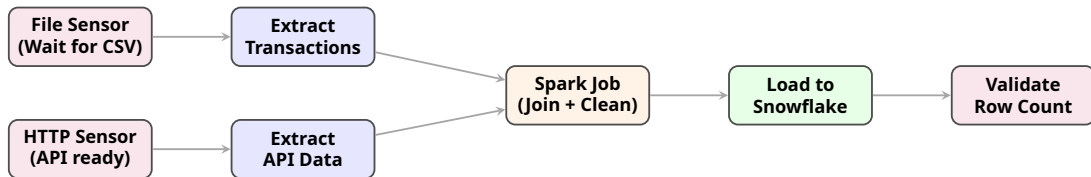
- Transactional database (PostgreSQL)
- External API (partner revenue)

Processing:

- Extract data from both sources
- Clean and join data using Spark
- Load results into a data warehouse (Snowflake)

SLA: Dashboard must be updated by 8 AM.

Pipeline architecture



Pipeline design principles

Key design decisions

- **Sensors** ensure data is ready before processing (wait for CSV file, check API availability)
- **Spark** handles the heavy transformation (not Airflow) - Airflow only orchestrates
- **Validation task** catches silent failures (e.g., empty results, row count mismatch)
- If validation fails, the DAG fails and alerts are sent to the on-call team

Why this architecture works

Each task has a single responsibility. If the Spark job fails, you can retry it without re-extracting data. If validation fails, you know immediately rather than discovering corrupt data later.

DAG implementation (simplified)

```
from airflow import DAG
from airflow.providers.apache.spark.operators.spark_submit import SparkSubmitOperator
from airflow.sensors.filesystem import FileSensor
from datetime import datetime, timedelta

default_args = {
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
    "email_on_failure": True,
}

with DAG(
    "daily_revenue_pipeline",
    start_date=datetime(2026, 1, 1),
    schedule_interval="0 2 * * *", # 2 AM daily
    catchup=False,
    default_args=default_args,
) as dag:
    wait_for_file = FileSensor(
        task_id="wait_for_transactions",
        filepath="/data/transactions_{{ ds }}.csv",
        poke_interval=60,
    )
    spark_job = SparkSubmitOperator(
        task_id="process_revenue",
        application="/jobs/revenue_processing.py",
        application_args=["{{ ds }}"],
    )
    wait_for_file >> spark_job
```

Operational considerations (1/2)

Monitoring and alerting

In production, observability is critical:

- Email alerts on failure
- Slack notifications for critical DAGs
- Dashboard showing DAG success rates
- SLA monitoring (must finish by 8 AM)

Handling late data

Scenario: CSV file arrives late from upstream.

Solution:

- FileSensor waits (with timeout)
- If timeout exceeded, DAG fails
- On-call engineer is paged and investigates the issue

Operational considerations (2/2)

Disaster recovery

Scenario: DAG failed yesterday, succeeded today. Dashboard shows missing data.

Solution:

- Manually trigger yesterday's run from Airflow UI
- Idempotent design ensures safe overwrite
- Dashboard is backfilled

This only works with idempotent pipelines.

Key operational principles

- Design for failure (retries, alerts, SLAs)
- Make pipelines idempotent for safe reruns
- Monitor continuously (logs, metrics, dashboards)
- Document recovery procedures for on-call engineers

Synthesis

What you should be able to do now

By now, you should understand

- Why cron is insufficient for modern data pipelines
- The difference between orchestration and computation
- How Airflow schedules DAGs (execution_date vs run time)
- The importance of idempotence for backfills
- Common anti-patterns that crash production systems
- How to design atomic, testable, observable pipelines

Three core principles

- (1) Airflow orchestrates, it does not compute.
- (2) Every task must be idempotent and retryable.
- (3) Observability is not optional in production.

Questions?

Contact: theo.burdinat1@epfedu.fr